AI Laboratory Report: Bayesian Networks & Autoregressive Language Models

Complete Solutions to Questions 1–14, Empirical CPTs, Normalisation Verification, and Reflections

Part I: From Probability to Language

Question 1: Why is the chain-rule autoregressive decomposition useful for generating text?

Modeling the joint probability distribution $P(X_1, X_2, \ldots, X_T)$ directly over whole sentences is intractable because the space of possible word sequences grows exponentially with length T (specifically $|V|^T$ for vocabulary size $|V|$). The chain-rule factorisation $P(X_1, \ldots, X_T) = P(X_1) \prod_{t=2}^{T} P(X_t \mid X_1, \ldots, X_{t-1})$ transforms a global high-dimensional distribution into a sequence of local, causal conditional distributions over a single vocabulary V at each step. This enables sequential ancestral sampling: we first sample $x_1 \sim P(X_1)$, condition on x_1 to sample $x_2 \sim P(X_2 \mid x_1)$, and repeat $x_t \sim P(X_t \mid x_1, \ldots, x_{t-1})$ token-by-token until an `<END>` token is produced.

Part II: A Bayesian Network for Text

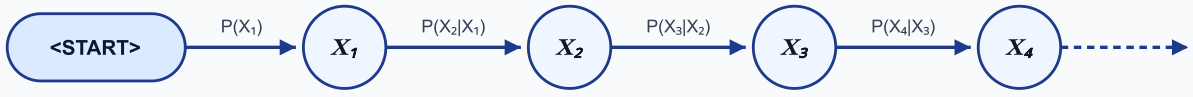


Figure 1: Directed Acyclic Graph (DAG) of a first-order Markov autoregressive language model.

Question 2: What independence assumption is being made by this network? Express using probability notation.

The network makes the **first-order Markov conditional independence assumption**: given the immediately preceding word X_{t-1}, the current word X_t is conditionally independent of all earlier words $X_1, X_2, \ldots, X_{t-2}$ in the sequence. In formal probability notation:

$$X_t \perp (X_1, X_2, \ldots, X_{t-2}) \mid X_{t-1} \iff P(X_t \mid X_1, X_2, \ldots, X_{t-1}) = P(X_t \mid X_{t-1}) \quad \text{for all } t \geq 2.$$

Consequently, the joint distribution over four tokens simplifies to $P(X_1, X_2, X_3, X_4) = P(X_1)\, P(X_2 \mid X_1)\, P(X_3 \mid X_2)\, P(X_4 \mid X_3)$.

Part III: Small Language Dataset & Preprocessing

Each of the 6 corpus sentences is converted to lower case, split into word tokens, and bracketed with `<START>` and `<END>` markers, yielding a vocabulary of $|V| = 12$ tokens (`<START>` , `the` , `cat` , `dog` , `sat` , `ran` , `on` , `to` , `mat` , `rug` , `park` , `<END>`):

```
1. <START> the cat sat on the mat <END>
2. <START> the cat sat on the rug <END>
3. <START> the dog sat on the mat <END>
4. <START> the dog ran to the park <END>
5. <START> the cat ran to the park <END>
6. <START> the dog sat on the rug <END>
```

Part IV: Constructing the Conditional Probability Table (Question 3)

Using Maximum Likelihood Estimation $P(w_j \mid w_i) = C(w_i, w_j) / \sum_k C(w_i, w_k)$ across the 6 training sentences (42 total bigram transitions), we construct the complete first-order Conditional Probability Table below. This includes the required contexts (`the` , `cat` , `dog` , `sat` , `ran`) as well as all remaining active tokens:

Current Word w_i	$\sum_k C(w_i, w_k)$	the	cat	dog	sat	ran	on	to	mat	rug	park	<END>
<START>	6	**6/6 (1.00)**	0	0	0	0	0	0	0	0	0	0
the *(req.)*	12	0	**3/12 (0.25)**	**3/12 (0.25)**	0	0	0	0	**2/12 (0.17)**	**2/12 (0.17)**	**2/12 (0.17)**	0
cat *(req.)*	3	0	0	0	**2/3 (0.67)**	**1/3 (0.33)**	0	0	0	0	0	0
dog *(req.)*	3	0	0	0	**2/3 (0.67)**	**1/3 (0.33)**	0	0	0	0	0	0
sat *(req.)*	4	0	0	0	0	0	**4/4 (1.00)**	0	0	0	0	0
ran *(req.)*	2	0	0	0	0	0	0	**2/2 (1.00)**	0	0	0	0
on	4	**4/4 (1.00)**	0	0	0	0	0	0	0	0	0	0
to	2	**2/2 (1.00)**	0	0	0	0	0	0	0	0	0	0
mat	2	0	0	0	0	0	0	0	0	0	0	**2/2 (1.00)**
rug	2	0	0	0	0	0	0	0	0	0	0	**2/2 (1.00)**
park	2	0	0	0	0	0	0	0	0	0	0	**2/2 (1.00)**

> **Question 3 (Continued): Identifying Zero-Probability Transitions**
>
> Every cell marked `0` in the table above is a zero-probability transition ($P(w_j \mid w_i) = 0$) because $C(w_i, w_j) = 0$ in the training dataset. Out of $11 \times 11 = 121$ possible transitions from the 11 non-terminal tokens to the 11 non-initial tokens, only **17 transitions are non-zero** and **104 transitions have zero probability**. Specifically: (1) after `the` , verbs/prepositions/markers (`the` , `sat` , `ran` , `on` , `to` , `<END>`) have probability 0; (2) after `cat` or `dog` , only `sat` (*2/3*) and `ran` (*1/3*) are non-zero, while all nouns, determiners, and prepositions have probability 0; (3) after `sat` , only `on` (*1.0*) occurs, so even grammatically valid English transitions like $P(to \mid sat) = 0$ or $P(<END> \mid sat) = 0$ receive zero probability; and (4) after `ran` , only `to` (*1.0*) occurs, making $P(on \mid ran) = 0$.

Part V & VI: Inspecting the LLM-Generated Code (Questions 4–7)

Question 4 (Where are the transition counts stored?): In `FirstOrderLanguageModel` (in `bn_language_models.py`), transition counts $C(w_i, w_j)$ are stored in the nested dictionary attribute `self.transition_counts = defaultdict(Counter)` . During `train()` , each consecutive pair `(w_prev, w_curr)` increments `self.transition_counts[w_prev][w_curr] += 1` .

Question 5 (Where is $P(X_t \mid X_{t-1})$ computed?): Immediately after counting in `train()` , the model computes total outgoing transitions `total_outgoing = sum(next_counts.values())` for each `w_prev` and populates `self.cpt[w_prev]` via the dictionary comprehension `{w_curr: count / total_outgoing for w_curr, count`

Part VII: Testing the Probability Model (Question 8)

Running `verify_normalization()` on our first-order CPT confirms that $\sum_v P(v \mid w) = 1.000000$ for all 11 conditioning tokens (`<START>` , `the` , `cat` , `dog` , `sat` , `ran` , `on` , `to` , `mat` , `rug` , `park`).

> **Question 8: If one of the totals is 0.87, what does this tell you about the implementation?**
>
> A row sum of `0.87` violates Kolmogorov's axiom of total probability ($\sum_{v \in V} P(v \mid w) = 1$), proving that the implementation has a bug and does not represent a valid conditional probability distribution. Common causes in code include: (1) dividing $C(w, v)$ by the unigram frequency $C(w)$ when sentence-final occurrences of w were included in the denominator without counting transitions to `<END>` in the numerator; (2) filtering out punctuation, stop-words, or low-frequency tokens *after* computing the normalization denominator; or (3) premature rounding/truncation of floating-point probabilities (e.g., rounding $1/6 \approx 0.1667$ down to 0.14).

Part VIII: Predicting the Next Word (Question 9)

Preceding Word X_t	Conditional Distribution $P(X_{t+1} \mid X_t)$	Most Probable Word $\arg\max_w P(w \mid X_t)$	Max Prob
`the`	`{'cat': 0.25, 'dog': 0.25, 'mat': 0.1667, 'rug': 0.1667, 'park': 0.1667}`	`cat` / `dog` *(exact tie)*	0.2500
`cat`	`{'sat': 0.6667, 'ran': 0.3333}`	`sat`	0.6667
`dog`	`{'sat': 0.6667, 'ran': 0.3333}`	`sat`	0.6667
`sat`	`{'on': 1.0000}`	`on`	1.0000
`ran`	`{'to': 1.0000}`	`to`	1.0000
`on`	`{'the': 1.0000}`	`the`	1.0000
`mat`	`{'<END>': 1.0000}`	`<END>`	1.0000

> **Question 9: Are the most probable predictions always the same as human expectations? What does this tell you?**
>
> No. While local collocations like `sat` → `on` look natural, predicting `cat` or `dog` as the most probable word after `the` even at the *end* of a sentence (such as *"the cat sat on the..."*, where a human expects a location noun like `mat` or `rug`) clashes with human expectations. This illustrates that a probability model has no innate semantic world knowledge or syntactic understanding: it only reflects empirical co-occurrence counts within its training corpus and structural Markov assumptions. Because `the` appears 6 times before subjects (`cat` / `dog`) and 6 times before objects (`mat` / `rug` / `park`), collapsing both roles into a single state $X_t = \text{the}$ causes the subject nouns ($3/12 = 0.25$ each) to outrank the object nouns ($2/12 \approx 0.167$ each).

Part IX: 20 Generated Sentences (First-Order Model)

Repeatedly sampling $X_t \sim P(X_t \mid X_{t-1})$ from `<START>` until `<END>` (seed 42) produces the following 20 sentences. Notice how the Markov property allows subject-object swaps (e.g., sentence #1 *"the dog sat on the cat sat on the rug"*):

```
 1. the dog sat on the cat sat on the rug               11. the cat sat on the dog sat on
the rug
 2. the cat sat on the cat ran to the park              12. the dog sat on the dog sat on
the rug
 3. the dog sat on the park                             13. the cat sat on the dog sat on
the mat
 4. the dog sat on the cat sat on the dog ran to the rug 14. the dog ran to the cat sat on
the mat
 5. the dog sat on the mat                              15. the cat ran to the mat
```

Part X: Deterministic vs. Probabilistic Generation (Question 10)

```
Mode A (Greedy - 5 runs, max_len=12):
  1. the cat sat on the cat sat on the cat sat on
  2. the cat sat on the cat sat on the cat sat on
  3. the cat sat on the cat sat on the cat sat on
the mat
  4. the cat sat on the cat sat on the cat sat on
  5. the cat sat on the cat sat on the cat sat on

Mode B (Sampling - 5 runs):
  1. the cat sat on the rug
  2. the dog ran to the park
  3. the cat sat on the dog sat on
  4. the park
  5. the dog sat on the mat
```

Question 10: Compare the two sets of generated sentences. Which mode produces more variation and why?

Mode B (Sampling) produces far greater variation (5 distinct sentences of varying lengths and structures), whereas **Mode A (Greedy)** produces zero variation across runs—generating the exact same infinite loop every time. Because Greedy decoding deterministically picks $arg\ max_w\ P(w \mid w_{prev})$, it always follows `<START>` → `the (1.0)` → `cat (0.25, tie-broken alphabetically)` → `sat (0.67)` → `on (1.0)` → `the (1.0)` → `cat (0.25)...`, trapping the model in a cycle where `mat / rug / park` ($P = 0.167$) and `<END>` are never reached. Sampling explores lower-probability transitions proportionally to their probability weights, enabling both diverse trajectories and proper termination.

Part XI & XII: Second-Order Bayesian Network (Question 11)

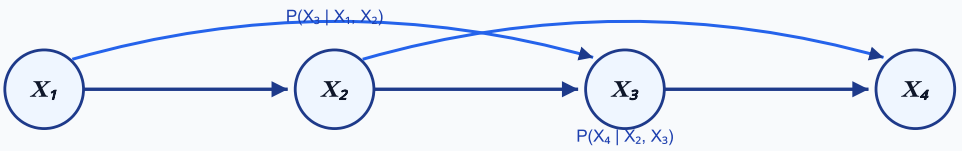


Figure 2: Second-Order Bayesian Network with factorisation $P(X_1, X_2, X_3, X_4) = P(X_1)\ P(X_2|X_1)\ P(X_3|X_1, X_2)\ P(X_4|X_2, X_3)$.

Question 11: How does the second-order model differ from the first-order model across the four dimensions?

1. **Graph Structure:** In the first-order DAG, each node X_t has only one parent ($X_{t-1} \rightarrow X_t$, a simple chain). In the second-order DAG, each node X_t ($t \geq 3$) has **two parents** ($X_{t-2} \rightarrow X_t \leftarrow X_{t-1}$), adding skip connections of length 2.
2. **Conditional Probability Table (CPT):** The first-order CPT is a 2D matrix indexed by a single prior word X_{t-1} with $O(|V|^2)$ entries. The second-order CPT is a 3D tensor indexed by ordered pairs (X_{t-2}, X_{t-1}) with $O(|V|^3)$ entries.
3. **Amount of Context Available:** First-order prediction conditions on 1 token of history; second-order prediction conditions on 2 tokens of history, disambiguating contexts like `(<START>, the)` vs. `(on, the)` vs. `(to, the)`.
4. **Amount of Data Needed:** Because the parameter space scales grows from $|V|^2$ to $|V|^3$, the second-order model requires exponentially more training data to observe sufficient trigram counts $C(w_{t-2}, w_{t-1}, w_t)$ and avoid severe sparsity.

Second-Order CPT Summary (Normalisation Verified: All 13 Observed Contexts Sum to 1.0)

```
Context (<START>, the) -> {'cat': 0.50, 'dog': 0.50}       Context (on, the)  -> {'mat':
0.50, 'rug': 0.50}
Context (to, the)      -> {'park': 1.00}                   Context (the, cat) -> {'sat':
0.67, 'ran': 0.33}
Context (the, dog)     -> {'sat': 0.67, 'ran': 0.33}       Context (cat, sat) -> {'on': 1.00}
```

Part XIII: Comparing the Two Models (Question 12)

Evaluation Metric	First-Order Model $P(X_t \mid X_{t-1})$	Second-Order Model $P(X_t \mid X_{t-2}, X_{t-1})$
Observed Contexts / Possible Contexts	11 observed / 11 possible (0 unobserved contexts)	13 observed / 110 possible (**97 zero-prob contexts**)
Non-Zero Parameters / Total CPT Cells	17 non-zero / 121 cells (104 zero cells; 86.0% sparse)	18 non-zero / 1,210 cells (1,192 zero cells; 98.5% sparse)
Diversity (20 Samples)	13 unique sentences (unbounded recursive length)	6 unique sentences (exact training distribution)
Qualitative Coherence & Examples	Low coherence: mixes subjects/objects & loops (e.g., *"the mat"*, *"the dog sat on the park"*, *"the dog sat on the cat sat on the rug"*).	High grammatical coherence: separates `(<START>, the)` from `(on, the)` and `(to, the)` (e.g., *"the cat sat on the mat"*, *"the dog ran to the park"*).

Question 12: Why does increasing context improve prediction yet make estimation harder from limited data?

Increasing context length from $k=1$ to $k=2$ improves prediction because it resolves structural ambiguity: in the first-order model, `the` conflates sentence-initial determiners and prepositional determiners, permitting ungrammatical outputs like *"the mat"* or *"sat on the park"*. Conditioning on two tokens separates `(<START>, the)` → `{cat, dog}`, `(on, the)` → `{mat, rug}`, and `(to, the)` → `{park}`. However, an n-gram CPT contains $|V|^n$ cells. As context increases, the number of conditioning histories grows exponentially ($11 \rightarrow 110$ contexts here), fragmenting a fixed corpus across vastly more cells and leaving 97 of 110 contexts with zero observations.

Part XV & Final Question: Role of the LLM & Bayesian Networks (Questions 13–14)

Question 13: Why is Approach B preferable to Approach A when constructing an intelligent system?

Specifying Intended Behaviour & Representation: A vague prompt like *"Write a Python language model"* (Approach A) leaves the mathematical model unspecified—an LLM might import `transformers`, build an unnormalized heuristic, or use character-level counts. Approach B explicitly defines the random variables, the conditional distribution $P(X_t \mid X_{t-1})$, MLE count estimation, and stochastic ancestral sampling. **Validating & Testing Probabilistic Invariants:** Because we know the formal Bayesian network specification, we can write automated invariant tests (such as $\sum_v P(v \mid w) = 1$ and boundary termination checks) to verify the code. **Distinguishing Implementation from Model:** Python dictionaries (`defaultdict(Counter)`) or neural networks are merely engineering mechanisms; the intelligent system is defined by its underlying probabilistic model.

Question 14: What did thinking of the language model as a Bayesian network give you?

- **Factorisation of the Joint Distribution & Representation of Dependencies:** Viewing text as a DAG $X_1 \rightarrow X_2 \rightarrow \ldots \rightarrow X_T$ explains how the intractable joint distribution $P(X_{1:T})$ decomposes into local CPTs via the chain rule.
- **Reasoning About Independence Assumptions & Context Scaling:** Graph topology makes the Markov assumption ($X_t \perp X_{1:t-2} \mid X_{t-1}$) explicit. When the first-order model generated *"the dog sat on the cat..."*, the graph structure immediately explained why: X_t had no edge from X_{t-2} (`on` vs. `<START>`), motivating the second-order v-structure $X_{t-2} \rightarrow X_t \leftarrow X_{t-1}$.
- **Principled Generation & Formal Verification:** It framed text generation as ancestral forward sampling over a DAG and provided strict mathematical invariants ($\sum_v P(v \mid \text{parents}) = 1$) to audit our implementation.